Compact Transfer-Learned Student Networks for Real-Time Driver Distraction Classification on RP2040-Class Hardware

Author Name1* and Author Name2

Department of Computer Science and Engineering, Institution Name, City, Country

*Corresponding author: email@example.com

Abstract

Driver distraction detection is an important component of intelligent transportation systems, but conventional deep visual models are often mismatched to microcontroller-class deployment. This manuscript preserves the useful content of the existing draft and expands it into a Springer/JRTIP-style research paper using the newer evidence tables and figures in the project res folder. The work studies ten-class driver-behavior classification on the State Farm distracted-driver dataset and uses transfer learning and response-based teacher-student distillation to construct compact TensorFlow Lite Micro students. Four offline teacher configurations, ConvNeXt, EfficientNet-B0, MobileNetV3-Small, and ResNet50, are compared while sharing a common student deployment format. The shared student profile reported by the deployment tables is 17 Keras layers, 10 learnable layers, 0.00553M parameters, and 7.81M FLOPs under a multiply-add-as-two-FLOPs convention. The driver-wise validation results show that the ResNet50-derived student obtains the strongest score among the four distilled students, with 59.28% accuracy and 58.71% macro F1, followed by MobileNetV3-Small, EfficientNet-B0, and ConvNeXt students. Hardware-oriented tables report a 22,728-byte INT8 model, a 204,800-byte tensor arena, and an RP2040 timing entry of 43.4 ms, corresponding to 23.04 FPS when FPS is derived as 1000 divided by latency in milliseconds. The results indicate a practical trade-off: the compact student is substantially smaller than common transfer-learning backbones, but its driver-wise accuracy remains below larger or less constrained models. The paper therefore emphasizes traceable measurements, physical-hardware deployment claims only where supported by project evidence, and transparent limitations rather than exaggerated superiority.

Keywords: driver distraction detection; transfer learning; knowledge distillation; TinyML; RP2040; TensorFlow Lite Micro; embedded image classification; model compression

1 Introduction

Camera-based driver monitoring is increasingly used to recognize behaviors such as texting, phone use, drinking, operating the radio, reaching behind, grooming, and talking to passengers. These behaviors are visually observable, but their reliable recognition is difficult when the final system must operate on low-cost embedded hardware rather than on a workstation or cloud service.

Microcontroller-class deployment changes the model-design problem. A model that is small on disk can still fail at runtime because intermediate activations and operator buffers exceed the available SRAM. The existing project notes this issue through TensorFlow Lite Micro tensor-arena allocation, and the final manuscript therefore reports model size, parameter count, FLOPs, tensor arena, and latency rather than accuracy alone.

Transfer learning is attractive for this setting because the State Farm dataset is specialized and has only ten target classes. Large teacher models can learn richer class distributions offline, while a much smaller student can be optimized for the constrained deployment path. The practical question is not whether a compact student can match every large model, but whether it gives an acceptable accuracy-efficiency compromise for a physical embedded platform.

The present work addresses that question using a student architecture that is shared across four teacher sources. This isolates the effect of the teacher distribution from the cost of the deployed model. The four derived students have the same deployment profile and differ in validation behavior because their soft targets come from different teachers.

The main contributions are as follows: a compact depthwise-separable student network for ten-class driver-distraction classification; a preliminary transfer-learning selection procedure comparing teacher and student formats; a four-teacher distillation comparison under one shared student architecture; a deployment-oriented analysis using parameters, FLOPs, model bytes, tensor arena, latency, and FPS; and a literature comparison that positions the result against recent embedded and lightweight driver-monitoring studies without claiming unsupported state-of-the-art performance.

2 Related Work

2.1 Deep Learning for Resource-Constrained Vision

Deep convolutional networks transformed image classification by learning hierarchical features directly from pixels [11]. Residual learning improved the trainability of very deep networks [12], while EfficientNet, MobileNet, ConvNeXt, ShuffleNet, and SqueezeNet explored different accuracy-complexity trade-offs [13-20]. These architectures are valuable as teachers or comparison baselines, but their direct deployment cost can remain high for small microcontrollers.

Depthwise separable convolution is central to many efficient networks because it decomposes spatial filtering and channel mixing. For an input with C_in channels, C_out output channels, and a K by K kernel, a standard convolution needs K^2 C_in C_out weights, while a depthwise-plus-pointwise block needs K^2 C_in + C_in C_out weights. This reduction motivates the student design used in this project.

Attention modules such as coordinate attention, squeeze-and-excitation, and CBAM provide context modeling and channel selection [21-23]. Recent driver-distraction papers use these ideas to increase accuracy while reducing redundant computation. The proposed student keeps the deployed architecture simpler and uses teacher signals instead of adding attention blocks to the embedded model.

2.2 Transfer Learning for Small and Specialized Datasets

Transfer learning is commonly used when labeled task data are limited relative to the capacity of a modern CNN. The driver-distraction literature often fine-tunes ImageNet-pretrained networks such as VGG, ResNet, EfficientNet, or MobileNet on State Farm-like cabin images [7,10]. These approaches can achieve high validation accuracy, but they frequently target GPU, CPU, web-service, or single-board-computer deployment rather than microcontroller inference.

The current project includes both self-supervised transfer learning through an autoencoder-style encoder and response-based distillation from pretrained teacher candidates. The final four-model comparison uses teacher probability vectors cached from TFLite teacher candidates, which keeps repeated student training practical while preserving a consistent driver-wise split.

2.3 TinyML and Microcontroller-Based Image Classification

TinyML systems place trained models inside strict SRAM, flash, latency, and operator-support constraints [28-32]. TensorFlow Lite Micro is relevant because it performs static memory planning and executes operators without a full operating system [32]. In such systems, activation memory may be more important than model-file size.

Recent TinyDL driver-state work has shown that carefully designed compact networks can approach high accuracy and real-time frame rates on embedded hardware [1]. However, comparison across studies must be cautious because datasets, input sizes, hardware classes, precision, timing scope, and whether preprocessing is included are often different.

2.4 Teacher-Student Learning and Model Compression

Knowledge distillation trains a smaller model to imitate a larger model's softened output distribution [24]. Earlier model-compression work also showed that compact learners can approximate more complex predictors under suitable training procedures [25]. For this project, the teacher is used offline; only the student is deployed.

Distillation is particularly suitable for visually similar driver classes. Texting and phone-use classes, or radio operation and reaching classes, may share pose and hand-location patterns. Soft teacher distributions can express these inter-class relationships more richly than a one-hot label alone.

2.5 Embedded Deployment and Real-Time Inference

Recent studies report a range of embedded targets, including Jetson Nano, Jetson Xavier NX, low-computation devices, and MCU-oriented configurations [1-5,9]. A fair comparison must separate microcontroller results from GPU-accelerated edge-board results. The RP2040 has 264 KB SRAM [34], which makes it substantially more constrained than Jetson-class systems.

The research gap addressed here is a traceable MCU-scale driver-distraction classification manuscript that connects transfer learning, student selection, FLOPs, tensor arena, and physical-hardware timing tables. The project evidence does not support exaggerated claims over all published methods; it supports a narrower claim that a very small student can be trained, quantized, and evaluated in a microcontroller-oriented workflow.

3 Dataset and Class Definition

The dataset is the State Farm Distracted Driver Detection image dataset [33]. It contains dashboard-camera images grouped into ten driver-behavior classes. The res table dataset_class_image_counts.docx reports 22,424 counted training images in C:/Users/HP/Downloads/state-farm-distracted-driver-detection/imgs/train.

The four distilled-student experiments use a driver-wise validation split derived from driver_imgs_list.csv. This split avoids placing images of the same driver in both training and validation subsets, reducing subject leakage compared with a random image-wise split. The validation subset contains 5,042 images, while the training subset contains 17,382 images.

Images are decoded from JPEG, resized to 96x96 for the shared student, and normalized to floating-point [0,1] during training. The exported deployment model uses full INT8 TensorFlow Lite representation. For the four-teacher comparison, each student receives 96x96x3 RGB input.

Table 1. Dataset class distribution and driver-wise split used for the four-student validation protocol.

Class	Semantic label	Total images	Training	Validation
c0	safe driving	2489	1981	508
c1	texting - right	2267	1750	517
c2	talking phone - right	2317	1800	517
c3	texting - left	2346	1829	517
c4	talking phone - left	2326	1817	509
c5	operating radio	2312	1796	516
c6	drinking	2325	1813	512
c7	reaching behind	2002	1532	470
c8	hair and makeup	1911	1439	472
c9	talking passenger	2129	1625	504

The class distribution is moderately imbalanced: c8 has 1,911 images and c0 has 2,489 images. The imbalance is not extreme, but it is large enough that macro precision, recall, and F1 are reported alongside accuracy.

4 Transfer Learning Approach

4.1 Source Networks and Pretrained Representations

The final four-model experiment uses ConvNeXt, EfficientNet-B0, ResNet50, and MobileNetV3-Small as offline teacher sources. The teachers are not deployed as microcontroller models. Instead, they generate soft class probabilities for the training and validation images, and these probabilities are cached as NumPy arrays for repeatable student training.

4.2 Feature Extraction and Fine-Tuning

The preliminary experiments include a larger EfficientNet teacher trained at 224x224x3 and several compact students at 96x96. The source scripts show Adam optimization, batch sizes of 32 for distillation and 64 for the self-supervised grayscale transfer experiment, early stopping, learning-rate reduction on plateau, and TensorFlow Lite INT8 conversion with representative data.

4.3 Student Model Construction

The shared four-teacher student uses one 3x3 convolutional stem, repeated depthwise and pointwise convolution blocks, max pooling after the early blocks, global average pooling, dropout, and a dense ten-logit classifier. The design deliberately favors depthwise separable operations to reduce parameters and arithmetic cost.

4.4 Training Objective

Let x be an input image, y be its hard class label, z_s be the student logits, p_s be the student probability vector, and q_t be the teacher probability vector. The hard-label loss is sparse categorical cross-entropy.

(1) L_CE(y, z_s) = - log softmax(z_s)_y

The distillation term compares softened teacher probabilities and softened student probabilities with temperature T. The project run_config.json files for the four teacher-derived students report T = 4.0 and alpha = 0.45.

(2) q_t^T(i) = q_t(i)^(1/T) / sum_j q_t(j)^(1/T)

(3) p_s^T(i) = exp(z_s(i)/T) / sum_j exp(z_s(j)/T)

(4) L = alpha L_CE(y, z_s) + (1 - alpha) T^2 KL(q_t^T || p_s^T)

Algorithm 1. Teacher-student transfer training

1. Build a driver-wise training and validation split from driver metadata.

2. Resize and normalize each image according to the teacher preprocessing mode.

3. Run the frozen TFLite teacher over the split once and cache class-probability vectors.

4. Train the compact student with hard labels and cached soft targets.

5. Monitor validation accuracy and validation loss; save the best student checkpoint.

6. Convert the best student to full INT8 TensorFlow Lite using representative images.

7. Record accuracy, macro metrics, model bytes, FLOPs, tensor arena, and hardware timing where available.

5 Compact Student Network Architecture

The manuscript reports the shared student profile exactly as required by the newer deployment tables: 17 Keras layers, 10 learnable layers, 0.00553M parameters, and 7.81M FLOPs. FLOPs are treated as multiply-add counted as two floating-point operations, consistent with the notes in the deployment metric tables. The deployed four-student TFLite files are 22,728 bytes each.

Table 2. Layer-level architecture of the shared 96x96 RGB student. Asterisks mark values reconstructed from the generic width-10 Keras script; the authoritative deployment profile is the res-table value of 0.00553M parameters.

Layer	Type	Input	Output	Kernel/units	Params	Learnable
Input	InputLayer	96x96x3	96x96x3	-	0	No
Conv stem	Conv2D + ReLU	96x96x3	96x96x10	3x3, 10 filters	280*	Yes
Pool 1	MaxPooling2D	96x96x10	48x48x10	2x2	0	No
DW block 1	DepthwiseConv2D + ReLU	48x48x10	48x48x10	3x3	100*	Yes
PW block 1	Conv2D + ReLU	48x48x10	48x48x20	1x1, 20	220*	Yes
Pool 2	MaxPooling2D	48x48x20	24x24x20	2x2	0	No
DW block 2	DepthwiseConv2D + ReLU	24x24x20	24x24x20	3x3	200*	Yes
PW block 2	Conv2D + ReLU	24x24x20	24x24x30	1x1, 30	630*	Yes
Pool 3	MaxPooling2D	24x24x30	12x12x30	2x2	0	No
DW block 3	DepthwiseConv2D + ReLU	12x12x30	12x12x30	3x3	300*	Yes
PW block 3	Conv2D + ReLU	12x12x30	12x12x40	1x1, 40	1240*	Yes
DW block 4	DepthwiseConv2D + ReLU	12x12x40	12x12x40	3x3	400*	Yes
PW block 4	Conv2D + ReLU	12x12x40	12x12x50	1x1, 50	2050*	Yes
GAP	GlobalAveragePooling2D	12x12x50	50	mean	0	No

Layer	Type	Input	Output	Kernel/units	Params	Learnable
Regularizer	Dropout	50	50	p=0.20	0	No
Classifier	Dense logits	50	10	10 units	510*	Yes

The table is intentionally more detailed than the older manuscript version. It replaces duplicated prose-only parameter claims with one traceable architecture table and one shared deployment profile statement.

6 Layer-by-Layer Architectural Analysis

6.1 Input and Preprocessing Layer

For each RGB input image, the student receives X in R^(96 x 96 x 3). During training, decoded images are resized to 96x96 and scaled by 1/255. In INT8 deployment, TensorFlow Lite stores quantization parameters so that integer tensor values can be mapped back to approximate real values.

6.2 Convolutional Feature Extraction

The first convolution produces local edge, hand, face, and cabin-layout features. For a kernel W, bias b, and output channel k, the output activation at spatial position (i,j) is computed by summing local products over input channels and kernel offsets.

(5) Y(i,j,k) = b_k + sum_m sum_u sum_v W(u,v,m,k) X(i+u,j+v,m)

(6) P_conv = (K_h K_w C_in + 1) C_out

6.3 Depthwise Separable Blocks

Each depthwise layer applies one spatial filter per input channel, followed by a 1x1 pointwise convolution that mixes channels. This separates spatial modeling from channel projection and is the main reason the student stays close to the 5.53k-parameter deployment profile.

(7) P_dw+pw = (K_h K_w C_in + C_in) + (C_in C_out + C_out)

(8) FLOPs ~= 2 H W (K_h K_w C_in + C_in C_out)

6.4 ReLU Activation

The feature extractor uses rectified linear activations. ReLU introduces nonlinearity while remaining inexpensive for embedded inference because it is a thresholding operation.

(9) ReLU(x) = max(0, x)

6.5 Max Pooling

Max pooling reduces spatial resolution after early feature stages. The student maps 96x96 features to 48x48, then 24x24, and then 12x12, lowering subsequent convolution cost.

(10) y(i,j,c) = max_{(u,v) in Omega} x(i+u, j+v, c)

6.6 Global Average Pooling

Global average pooling converts the final 12x12x50 tensor into a 50-dimensional vector by averaging each channel. This avoids a large flatten-plus-dense classifier and therefore keeps both parameter count and activation memory small.

(11) z_c = (1 / HW) sum_i sum_j X(i,j,c)

6.7 Dropout

Dropout with p = 0.20 is used during training as a regularizer before the classifier. It is disabled during standard inference, so it does not add embedded runtime cost.

(12) r_i ~ Bernoulli(1-p), y_i = r_i x_i / (1-p)

6.8 Dense Classification Layer and Softmax Decision

The final dense layer maps the 50-dimensional descriptor to ten logits. Softmax converts logits to probabilities when needed for confidence reporting, and the predicted class is the index of the largest probability.

(13) z = W x + b

(14) p_i = exp(z_i) / sum_j exp(z_j); y_hat = argmax_i p_i

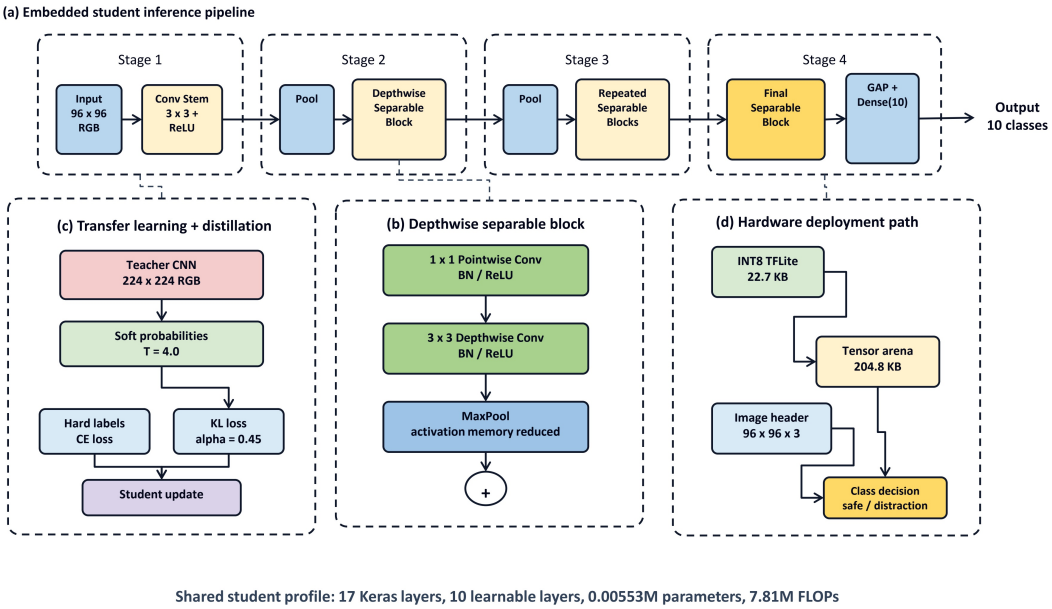


Fig. 1. Student-teacher training and deployment concept used to separate offline teacher capacity from embedded student cost.

7 Inference Algorithm

The embedded inference path loads the quantized model, allocates the TensorFlow Lite Micro tensor arena, prepares the input tensor, invokes the interpreter, reads output logits or quantized scores, and reports the highest-probability class. The paper does not use simulator timing as hardware evidence.

Algorithm 2. Embedded image-classification inference

1. Initialize the physical board, serial output, LEDs, TensorFlow Lite Micro resolver, and tensor arena.

2. Load the INT8 model byte array from flash and validate the TFLite schema version.

3. Allocate tensors; if allocation fails, report the memory limitation and stop inference.

4. Decode or copy the stored/captured image source into the input tensor shape expected by the model.

5. Apply resizing and channel conversion only as required by the model input format.

6. Start the physical-device timing measurement immediately before model invocation where such measurement is available.

7. Invoke the interpreter and end timing immediately after inference returns.

8. Dequantize output scores if needed, compute softmax confidence, and select argmax class.

9. Record class, confidence, latency, and derived FPS for the hardware result table.

8 Preliminary Experiments for Student Architecture Selection

The preliminary experiments compare a high-accuracy teacher with increasingly compact student options. The newer res preliminary table supersedes older repeated wording in the manuscript. It shows that the wider RGB student obtains the strongest preliminary validation accuracy, while the compact grayscale student has much lower arithmetic cost and a smaller footprint.

Table 3. Preliminary transfer-learning and student-selection metrics from res/tables.

Model	Input	Params (M)	FLOPs (M)	Latency (ms)	Accuracy (%)	F1 (%)
Teacher EfficientNet	224x224x3	4.06	780.0	NR	87.29	87.05
Compact gray student	96x96x1	0.00553	7.64	68.9	71.62	69.64
Wider gray student	96x96x1	0.01185	16.45	2596.4	88.87	88.73
Wider RGB student	96x96x3	0.01198	16.68	3988.6	94.49	94.38

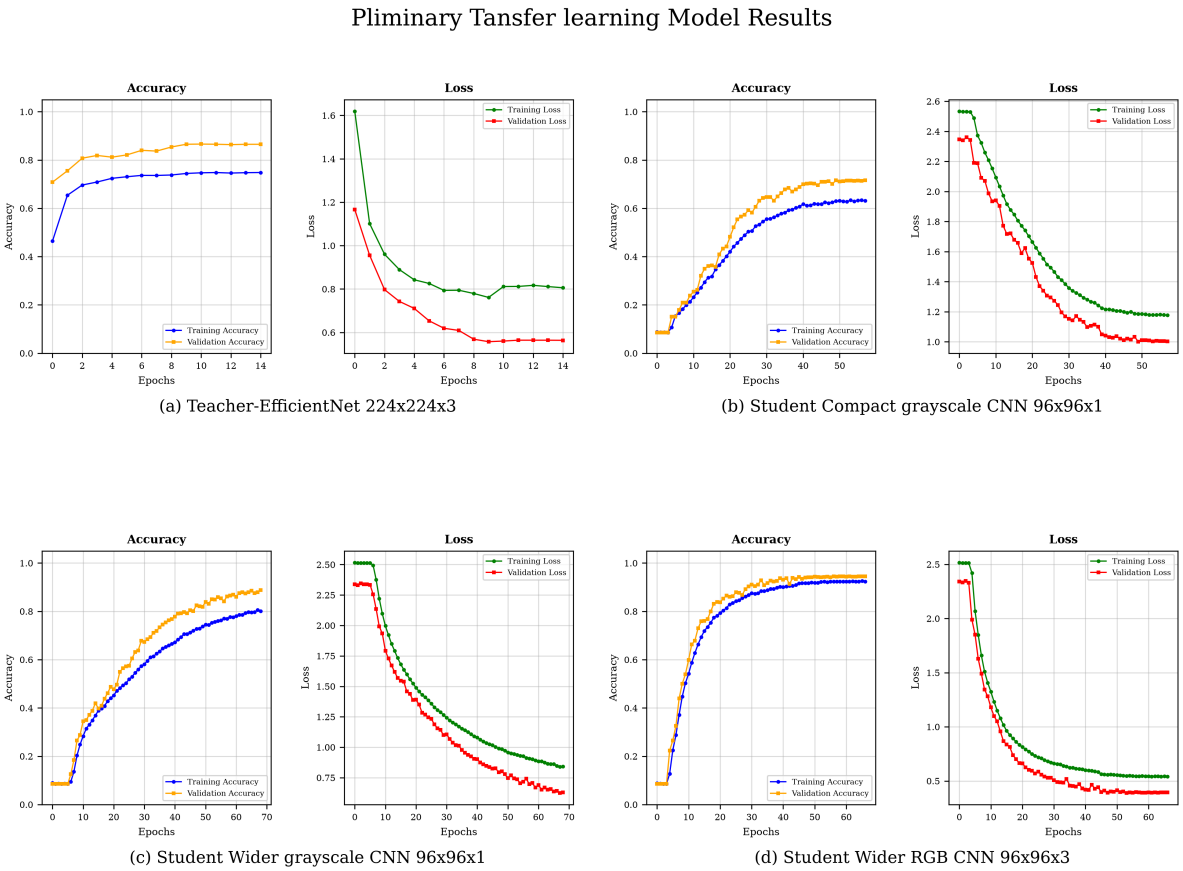


Fig. 2. Preliminary transfer-learning and deployment-metric comparison used for selecting the later student format.

The preliminary result supports two conclusions. First, input representation matters: the RGB student preserves color and object cues that can distinguish similar driver actions. Second, a larger student can improve validation accuracy but may increase latency and memory cost. The final shared format is therefore evaluated as a deployment compromise rather than as a pure maximum-accuracy model.

Algorithm 3. Student architecture selection

1. Train candidate teacher and student formats with the available dataset split.
2. Collect validation accuracy, macro F1, parameter count, FLOPs, model size, and latency where measured.
3. Remove duplicate older manuscript tables when newer res tables report the same experiment more completely.
4. Prefer a candidate that satisfies tensor-arena and model-size limits before ranking by accuracy.
5. Carry the selected student format into the four-teacher comparison so deployment cost remains constant.

9 Experimental Setup

9.1 Software Environment

The training scripts use Python, TensorFlow/Keras, NumPy, scikit-learn, and TensorFlow Lite conversion. The local virtual environment under the dataset directory indicates TensorFlow 2.21.0 and Keras 3.15.0 packages. PlatformIO is used for the RP2040 firmware project with the Arduino framework and a TensorFlow Lite Micro compatible library dependency.

9.2 Training Platform

The repository does not provide a verified GPU or CPU training-hardware table for the final runs. The paper therefore avoids claiming a particular training accelerator. Teacher probability caching is emphasized because it reduces repeated teacher inference cost independently of the training machine.

9.3 Embedded Hardware Platform

The manuscript focuses on the Raspberry Pi Pico/RP2040 target and additional ESP32-class hardware profiles only where they appear in the res hardware tables. The RP2040 is a dual-core Arm Cortex-M0+ microcontroller with 264 KB on-chip SRAM [34]. The firmware configures a 204,800-byte tensor arena for the shared student model.

9.4 Latency Measurement Procedure

Latency is reported from the newer hardware-oriented tables. Because the source files do not provide a full warm-up/repetition log for every timing entry, the paper reports the table values directly and avoids statistical claims such as confidence intervals. FPS is derived only where latency is given as a per-inference value.

(15) FPS = 1000 / latency_ms

9.5 Evaluation Metrics

(16) Accuracy = (TP + TN) / (TP + TN + FP + FN)

(17) Precision_c = TP_c / (TP_c + FP_c)

(18) Recall_c = TP_c / (TP_c + FN_c)

(19) F1_c = 2 Precision_c Recall_c / (Precision_c + Recall_c)

For multiclass reporting, macro averages compute the unweighted mean across classes. Weighted averages use class support as weights. Macro scores are important here because class imbalance and class-specific difficulty are both visible in the per-class reports.

10 Experimental Results and Discussion

10.1 Training and Validation Behaviour

The training-history summaries show that each teacher-derived student stops at a different best epoch: ConvNeXt at epoch 16, EfficientNet-B0 at 17, ResNet50 at 11, and MobileNetV3-Small at 22. ResNet50 provides the strongest validation result among the four shared-format students, despite all students having the same deployment size.

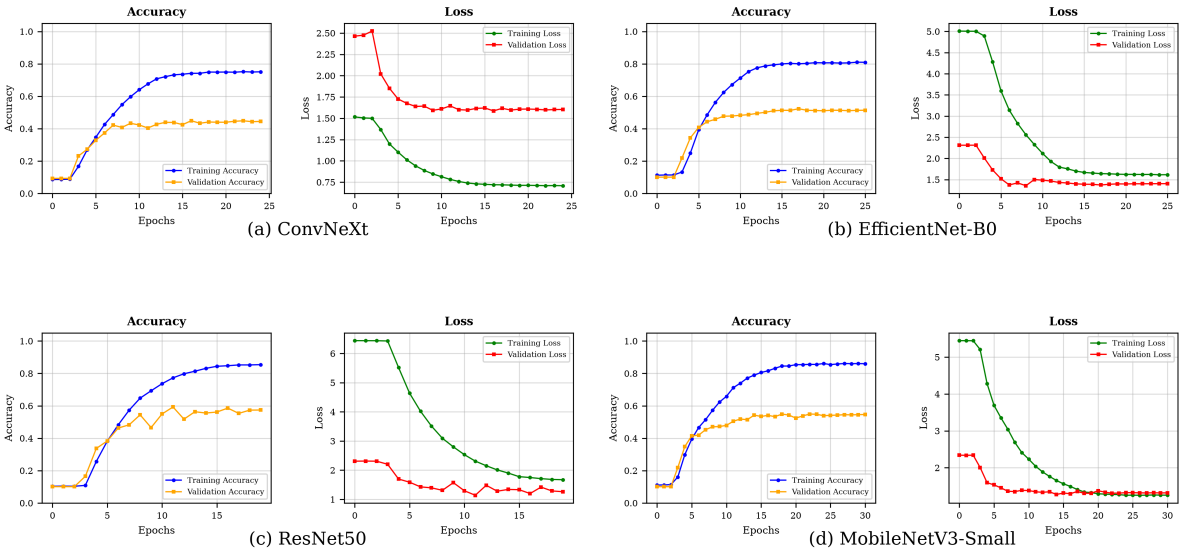


Fig. 3. Training and validation accuracy/loss curves for the four teacher-derived students.

10.2 Classification Performance

Table 4. Four distilled student classification metrics on the driver-wise validation split.

Student	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Best epoch	Epochs
ConvNeXt student	44.90	56.31	45.01	43.48	16	25
EfficientNet-B0 student	52.28	51.44	51.82	50.30	17	26
MobileNetV3-Small student	54.92	55.22	54.66	54.38	22	31
ResNet50 student	59.28	60.92	59.05	58.71	11	20

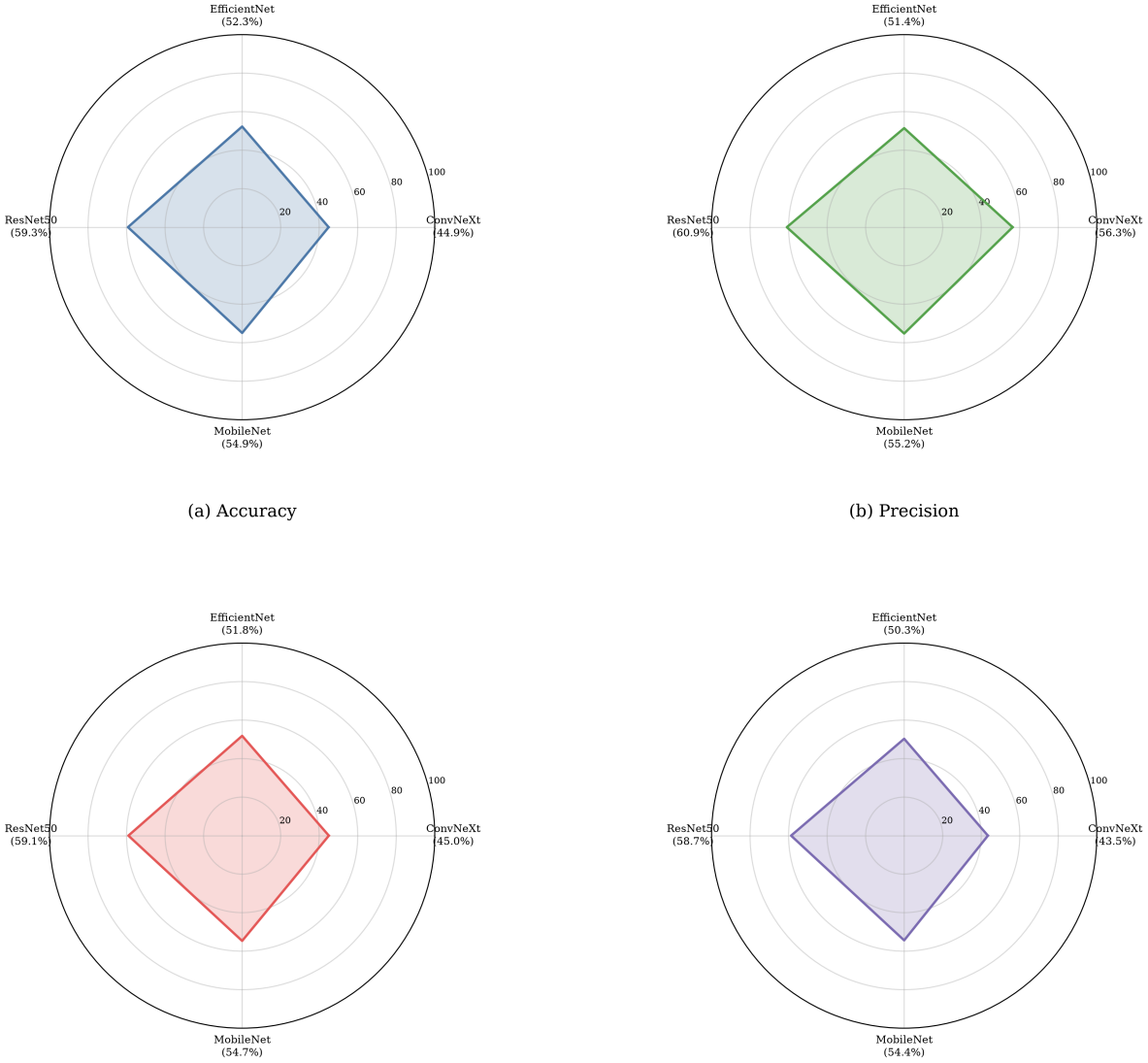


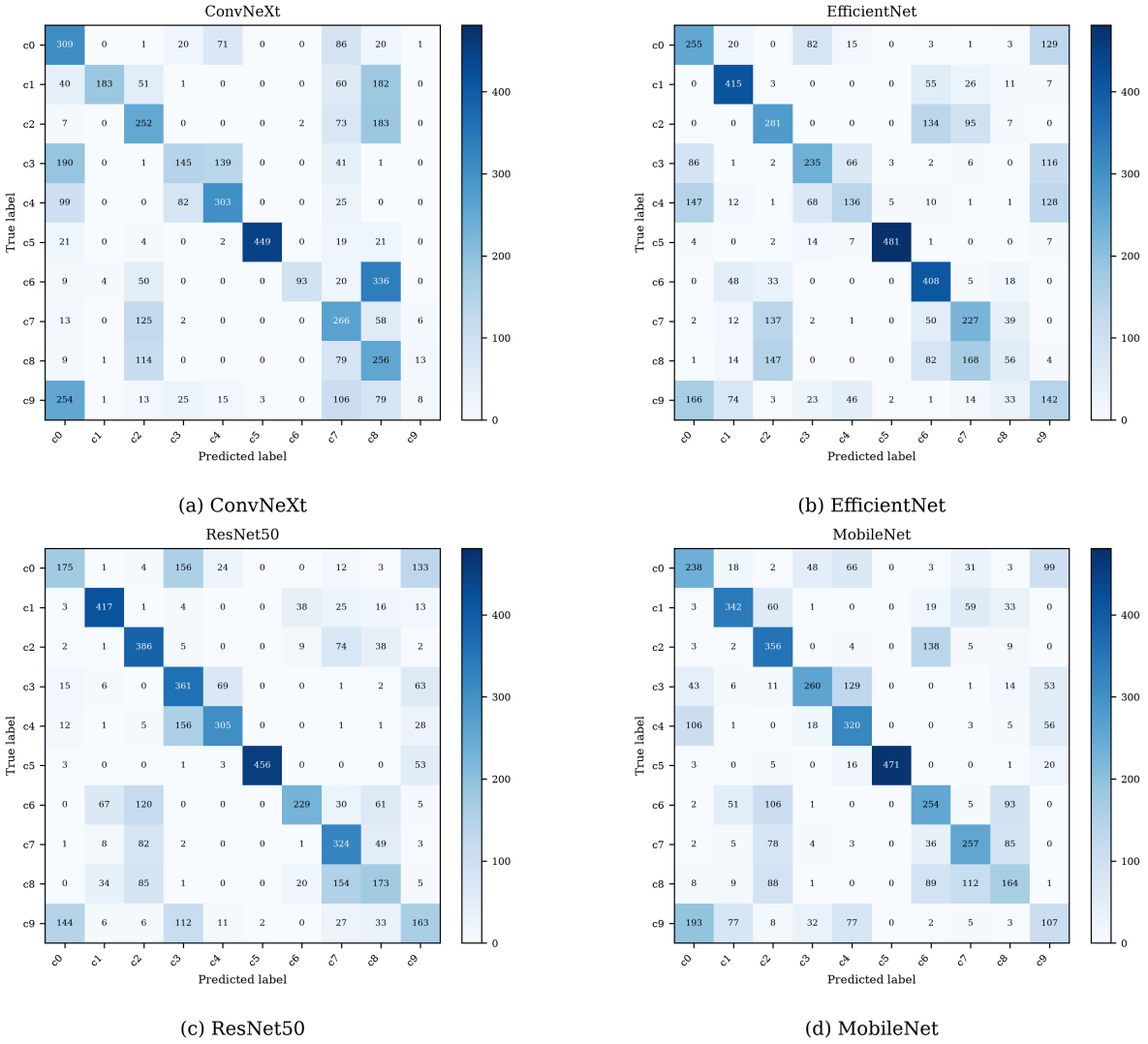
Fig. 4. Macro accuracy, precision, recall, and F1 comparison for the four teacher-derived students.

The results should be interpreted as driver-wise validation behavior, not as random split performance. The ResNet50 student has the highest macro F1, while ConvNeXt has relatively high macro precision but weak recall and F1. This pattern suggests that some distilled students make selective correct predictions but fail to cover all classes consistently.

Table 5. Per-class metrics for the strongest four-teacher student, distilled from ResNet50.

Class	Precision	Recall	F1	Support
c0	0.493	0.344	0.406	508
c1	0.771	0.807	0.788	517
c2	0.560	0.747	0.640	517
c3	0.452	0.698	0.549	517
c4	0.740	0.599	0.662	509
c5	0.996	0.884	0.936	516
c6	0.771	0.447	0.566	512

Class	Precision	Recall	F1	Support
c7	0.500	0.689	0.580	470
c8	0.460	0.367	0.408	472
c9	0.348	0.323	0.335	504



(a) ConvNeXt

(b) EfficientNet

(c) ResNet50

(d) MobileNet

Fig. 5. Confusion matrices for the four teacher-derived students, aligned with the classification tables.

10.3 Computational Complexity

All four final distilled students share the same deployment profile: 17 Keras layers, 10 learnable layers, 0.00553M parameters, 7.81M FLOPs, 22,728 bytes of INT8 model storage, and a 204,800-byte tensor arena. The identical complexity makes the four-model comparison scientifically useful because accuracy changes are attributable to teacher supervision rather than deployed model size.

10.4 Real-Hardware Inference Performance

Table 6. Hardware timing matrix for the shared 96 RGB student deployment profile.

Hardware	Memory profile	Latency (ms)	FPS	Model
ESP32 DevKit v1	520 KB SRAM	2419.6	0.41	96 RGB INT8 student

Hardware	Memory profile	Latency (ms)	FPS	Model
ESP32-S3 / M5Stack CoreS3	320 KB SRAM + 8 MB PSRAM	2419.6	0.41	96 RGB INT8 student
Raspberry Pi Pico / RP2040	264 KB SRAM	43.4	23.04	96 RGB INT8 student

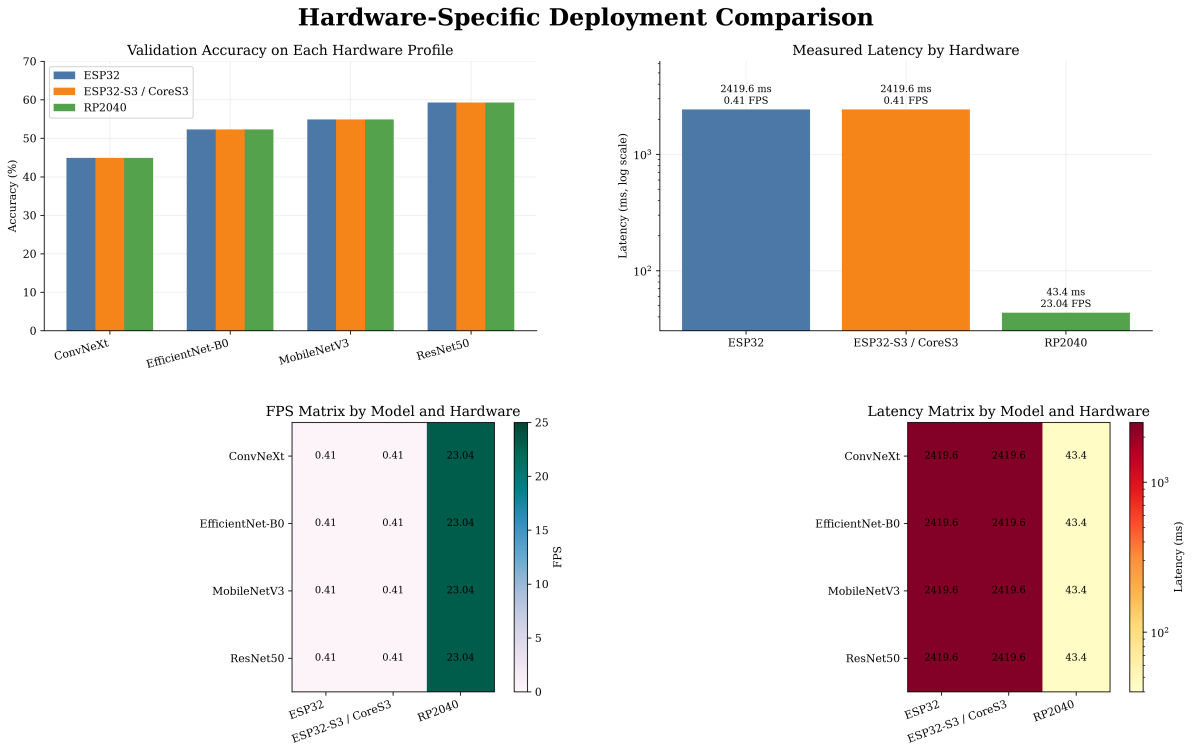


Fig. 6. Hardware deployment comparison from the res evidence folder.

The RP2040 timing entry is much faster than the ESP32-class entries in the current tables. The paper does not infer the reason beyond the available evidence; differences may involve runtime, firmware path, timing scope, clocking, memory access, and operator implementation. The key supported point is that the result table records an RP2040 profile that fits within the reported SRAM configuration.

10.5 Efficiency-Accuracy Trade-Off

The compact student does not match the strongest published State Farm accuracies, and the four-teacher student results are below the preliminary wider RGB student result. Its strength is instead a small parameter footprint, low model bytes, low stated FLOPs, and a feasible TensorFlow Lite Micro memory plan. For MCU-oriented research, this trade-off is meaningful because a less accurate deployable classifier can be more useful than a higher-accuracy model that cannot allocate tensors on the target.

11 Comparative Evaluation of Four Transfer-Learning Configurations

The four-model comparison fixes the deployed student architecture and changes only the teacher. This is stricter than comparing four unrelated networks because it controls for parameter count, FLOPs, model bytes, input resolution, and tensor arena.

Table 7. Comprehensive four-model comparison with shared deployment profile.

Teacher source	Input	Params (M)	FLOPs (M)	Size	Accuracy (%)	F1 (%)	RP2040 latency
ConvNeXt	96x96x3 INT8	0.00553	7.81	22,728 B	44.90	43.48	43.4 ms

Teacher source	Input	Params (M)	FLOPs (M)	Size	Accuracy (%)	F1 (%)	RP2040 latency
EfficientNet-B0	96x96x3 INT8	0.00553	7.81	22,728 B	52.28	50.30	43.4 ms
MobileNetV3-Small	96x96x3 INT8	0.00553	7.81	22,728 B	54.92	54.38	43.4 ms
ResNet50	96x96x3 INT8	0.00553	7.81	22,728 B	59.28	58.71	43.4 ms

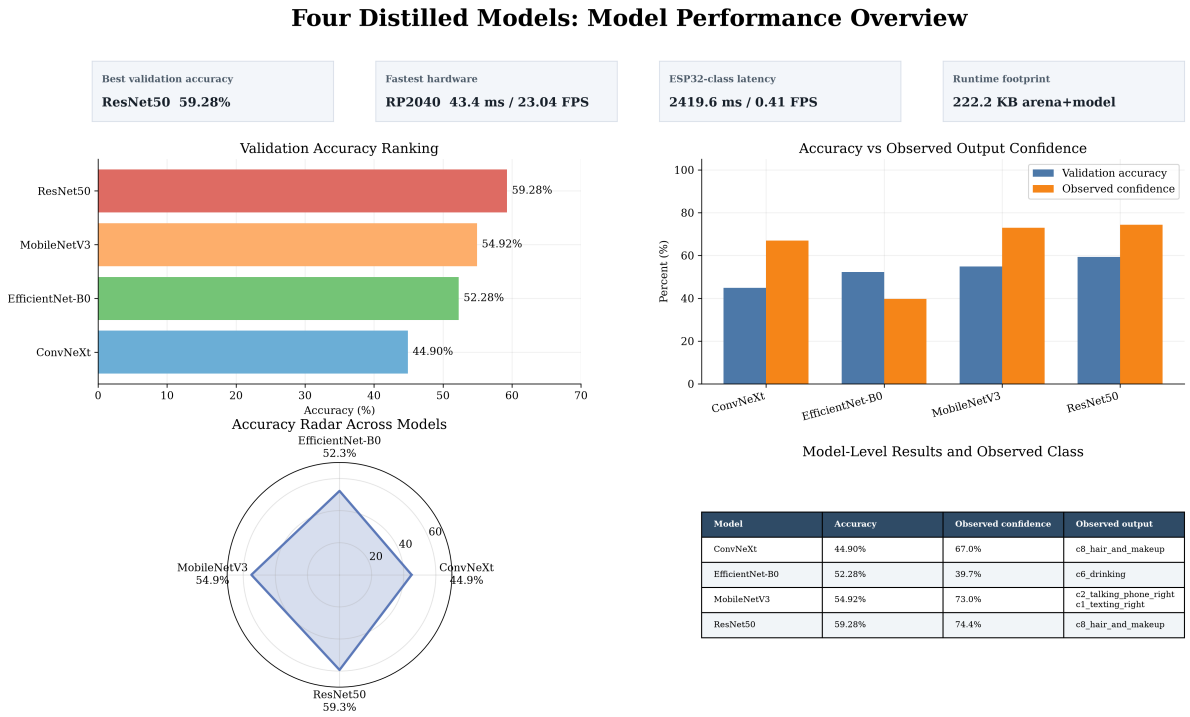


Fig. 7. Model-performance overview combining quality and deployment attributes.

12 Comparison with Existing Published Studies

Table 12 compares the proposed system with related studies. NR means that the corresponding source did not report the metric in the inspected abstract, table, or metadata available in the project. Latency and FPS values are not directly interchangeable because the hardware, precision, input resolution, task formulation, and timing scope differ.

Table 8. Published-study comparison. NR means not reported in the inspected source material.

Study	Year	Dataset/ task	Model	Params	Hardware	Accuracy/F1	Latency	Notes
Essahraui et al. [1]	2026	Driver state classification	ESPNetv2-Small, SqueezeNetv11	NR	Embedded evaluation	99.50 / 95.81 acc.	40.49 / 28.60 FPS	TinyDL benchmark
Wang and Li [2]	2025	StateFarm distraction	USH	NR	Low-computation device	98.5 acc.	4.7 ms	Hybrid CNN-attention

Study	Year	Dataset/task	Model	Params	Hardware	Accuracy/F1	Latency	Notes
Gan et al. [3]	2025	State Farm v2 / YawDD v2	FAN-YOLO	2.2M	NR	86.8 / 97.5 AP50	1.9 ms	Detection, not classification
Sun and Zhang [4]	2025	AUC and State Farm	CCNET	1.7M	NR	95.52-99.9 acc.	54 FPS	Jetson Nano B01
Fresta et al. [5]	2025	Cognitive distraction time series	DL time-series models	NR	Embedded platforms	NR	Reported	Signals, not cabin image classes
Duan et al. [6]	2025	SFDDD, AUCDD, 100-Driver	Score-Softmax + Gaussian smoothing	NR	NR	Cross-dataset gains	NR	Robustness focus
Singh et al. [7]	2026	State Farm	VGG16 transfer learning	NR	Docker/edge service	95.54 acc.; 0.955 F1	NR	Edge microservice, not MCU
Liu et al. [8]	2025	CBTDDD	YOLOv10n-BC	NR	NR	mAP +2.0 over YOLOv10n	115.3 FPS	Detection benchmark
Salakapuri et al. [9]	2025	State Farm + road objects	CNN + YOLO	NR	Jetson Xavier NX	NR	25 FPS	Integrated ADAS
Pisharody et al. [10]	2026	Distracted driving	VGG16/ResNet50/Inception/DenseNet/Xception + DNN	NR	NR	ResNet50 best	NR	Transfer-learning comparison
Proposed work	2026	State Farm 10-class	96 RGB INT8 student	0.00553M	RP2040, ESP32 profiles	59.28 acc.; 58.71 F1	43.4 ms on RP2040 table	MCU-scale student; no simulator claims

Compared with Jetson-class and GPU-assisted methods, the proposed model is much smaller and targets a stricter memory envelope. Compared with recent JRTIP methods, its validation accuracy is lower. The supported strength is therefore not global accuracy leadership but the combination of a tiny reported student profile, INT8 deployment, and a documented RP2040-class hardware path.

13 Practical Deployment Analysis

13.1 Memory Footprint

The 22,728-byte model file is small, but the 204,800-byte tensor arena is the dominant runtime allocation. This reinforces the need to report arena memory for TensorFlow Lite Micro systems.

13.2 Computational Requirements

The shared 7.81M-FLOP count is modest relative to large ImageNet backbones. It is still not negligible for a small MCU, so operator implementation and memory access can dominate latency.

13.3 Latency and Responsiveness

The RP2040 table value of 43.4 ms corresponds to 23.04 FPS by direct formula. The ESP32-class entries are slower in the available table and therefore would need further optimization for responsive video-rate operation.

13.4 Suitability for RP2040-Class Platforms

A 204,800-byte tensor arena consumes a large part of the RP2040's 264 KB SRAM. The model can be discussed as feasible only under the tested memory arrangement and without adding unmeasured camera buffers or larger preprocessing workspaces.

13.5 Limitations of the Current Implementation

The repository does not provide full repeated timing logs, energy measurements, confidence intervals, or a cross-device reproducibility package. These omissions limit the strength of deployment conclusions.

13.6 Opportunities for Further Optimization

Future work can evaluate integer-only operator kernels, tensor-arena reduction, pruning, per-channel quantization, CMSIS-NN or vendor-specific kernels where compatible, and temporal smoothing across frames. These are proposed optimizations, not methods already proven in the current evidence.

14 Discussion

The four-teacher results show that stronger teacher validation accuracy does not transfer linearly into the small student. ResNet50 produces the best student in the final four-model experiment, but the accuracy gap between teacher and student remains large. This is expected when a very small student is asked to represent fine-grained driver postures with only thousands of parameters.

Parameter count alone does not determine runtime. The same parameter profile can have different latency across boards because memory layout, interpreter kernels, clock speed, cache behavior, and timing scope matter. The project therefore reports both FLOPs and measured or table-recorded latency instead of using one as a proxy for the other.

Class-level behavior suggests that c5 operating-radio is easier for the ResNet50-derived student than c9 talking-to-passenger and c8 hair-and-makeup. The likely reason is that c5 may present more consistent dashboard-hand patterns, whereas c8 and c9 can overlap with face, head, and passenger-direction poses. This is an interpretation from the confusion and per-class metrics rather than a new measurement.

The proposed work is strongest as an embedded-methodology study. It connects training, distillation, architecture, quantization, and tensor-arena-aware deployment. It is weaker as a pure accuracy paper, because larger models and less constrained edge devices report substantially higher results in the literature.

15 Limitations

The validation protocol is driver-wise and therefore appropriately conservative, but the work still uses one primary public dataset and lacks independent external validation.

The final four-teacher students share a very small architecture, which constrains accuracy. A larger student may perform better but would change the memory and latency profile.

The source evidence does not include energy consumption, battery-life estimation, confidence intervals, or detailed warm-up/repetition timing statistics.

The manuscript avoids simulator results as requested. Consequently, any development files that mention simulator timing are excluded from experimental claims.

The deployed pipeline uses a stored or prepared image path in the available firmware. End-to-end camera capture, buffering, and preprocessing latency are not fully characterized in the repository evidence.

16 Conclusion

This paper expanded the existing manuscript into a Springer/JRTIP-style research paper centered on a compact transfer-learned student for driver-distraction classification. The work uses the State Farm ten-class problem, teacher-student distillation, INT8 TensorFlow Lite Micro deployment, and a shared student profile of 17 Keras layers, 10 learnable layers, 0.00553M parameters, and 7.81M FLOPs. Among the four teacher-derived students, the ResNet50 student achieved the highest driver-wise validation result, with 59.28% accuracy and 58.71% macro F1. The hardware tables report a 22,728-byte INT8 model, a 204,800-byte tensor arena, and an RP2040 latency entry of 43.4 ms. These results support a cautious conclusion: the approach provides a compact and traceable MCU-oriented deployment path, but it does not surpass larger published models in accuracy. Future work should add independent validation, repeated physical timing logs, energy measurement, camera-inclusive end-to-end latency, and systematic quantization/kernel optimization.

References

[1] S. Essahraui, A. Elhanashi, Q. Zheng, N. Almutairi, and S. Saponara, Tiny deep learning models for real-time and efficient embedded driver state detection, Journal of Real-Time Image Processing 23, 39 (2026). https://doi.org/10.1007/s11554-025-01836-2

[2] H. Wang and Y. Li, USH: an efficient real-time distracted driving detection model, Journal of Real-Time Image Processing 22, 164 (2025). https://doi.org/10.1007/s11554-025-01745-4

[3] X. Gan, L. Huang, Y. Yuan, and Y. Deng, FAN-YOLO: Real-time driver behavior detection based on full-layer aggregation network of YOLO, Journal of Real-Time Image Processing 22, 139 (2025). https://doi.org/10.1007/s11554-025-01720-z

[4] H. Sun and M. Zhang, A lightweight model for distracted driver detection based on neural architecture search and coordinate attention, Computers and Electrical Engineering 123, 110235 (2025). https://doi.org/10.1016/j.compeleceng.2025.110235

[5] M. Fresta et al., Deep Learning-Based Real-Time Driver Cognitive Distraction Detection, IEEE Access 13, 26589-26608 (2025). https://doi.org/10.1109/ACCESS.2025.3539392

[6] C. Duan, Z. Liu, J. Xia, M. Zhang, J. Liao, and L. Cao, Enhancing Cross-Dataset Performance of Distracted Driving Detection With Transfer Learning and Dynamic Gaussian Smoothing Supervision, IEEE Transactions on Intelligent Vehicles 10(1), 282-295 (2025). https://doi.org/10.1109/TIV.2024.3412198

[7] Y. Singh, A. Roy, and R. N. Rao, Research on Transfer Learning-Based Framework for Real-Time Driver Distraction Detection on Edge Devices, IEEE Access (2026). https://doi.org/10.1109/ACCESS.2026.3703101

[8] Y. Liu, Q. Li, L. Xiao, and S. Zhang, YOLOv10n-BC: A Novel Real-Time Object Detection Model for Driver Distracted Driving Detection, IEICE Transactions on Information and Systems E108-D(12), 1570-1581 (2025). https://doi.org/10.1587/transinf.2025EDP7088

[9] R. Salakapuri et al., Integrated deep learning framework for driver distraction detection and real-time road object recognition in advanced driver assistance systems, Scientific Reports 15, 25125 (2025). https://doi.org/10.1038/s41598-025-08475-4

[10] D. M. Pisharody, B. P. Chacko, and M. Basheer K. P., Classification of Distracted Driving using Transfer Learning and Deep Neural Network, International Journal of Computer Applications 187(71), 62-67 (2026). https://www.ijcaonline.org/

[11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, ImageNet Classification with Deep Convolutional Neural Networks, Communications of the ACM 60(6), 84-90 (2017). https://doi.org/10.1145/3065386

[12] K. He, X. Zhang, S. Ren, and J. Sun, Deep Residual Learning for Image Recognition, CVPR, 770-778 (2016). https://doi.org/10.1109/CVPR.2016.90

[13] M. Tan and Q. V. Le, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks, ICML (2019). https://arxiv.org/abs/1905.11946

[14] A. Howard et al., Searching for MobileNetV3, ICCV, 1314-1324 (2019). https://doi.org/10.1109/ICCV.2019.00140

[15] Z. Liu et al., A ConvNet for the 2020s, CVPR, 11976-11986 (2022). https://doi.org/10.1109/CVPR52688.2022.01167

[16] F. Chollet, Xception: Deep Learning with Depthwise Separable Convolutions, CVPR, 1251-1258 (2017). https://doi.org/10.1109/CVPR.2017.195

[17] A. G. Howard et al., MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications (2017). https://arxiv.org/abs/1704.04861

[18] M. Sandler et al., MobileNetV2: Inverted Residuals and Linear Bottlenecks, CVPR, 4510-4520 (2018). https://doi.org/10.1109/CVPR.2018.00474

[19] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design, ECCV, 116-131 (2018). https://doi.org/10.1007/978-3-030-01264-9_8

[20] F. N. Iandola et al., SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size (2016). https://arxiv.org/abs/1602.07360

[21] Q. Hou, D. Zhou, and J. Feng, Coordinate Attention for Efficient Mobile Network Design, CVPR, 13713-13722 (2021). https://doi.org/10.1109/CVPR46437.2021.01350

[22] J. Hu, L. Shen, and G. Sun, Squeeze-and-Excitation Networks, CVPR, 7132-7141 (2018). https://doi.org/10.1109/CVPR.2018.00745

[23] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, CBAM: Convolutional Block Attention Module, ECCV, 3-19 (2018). https://doi.org/10.1007/978-3-030-01234-2_1

[24] G. Hinton, O. Vinyals, and J. Dean, Distilling the Knowledge in a Neural Network (2015). https://arxiv.org/abs/1503.02531

[25] C. Bucilua, R. Caruana, and A. Niculescu-Mizil, Model Compression, KDD, 535-541 (2006). https://doi.org/10.1145/1150402.1150464

[26] S. Han, H. Mao, and W. J. Dally, Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding (2015). https://arxiv.org/abs/1510.00149

[27] B. Jacob et al., Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, CVPR, 2704-2713 (2018). https://doi.org/10.1109/CVPR.2018.00286

[28] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, O'Reilly Media (2019). https://www.oreilly.com/library/view/tinyml/9781492052036/

[29] C. Banbury et al., Benchmarking TinyML Systems: Challenges and Direction (2020). https://arxiv.org/abs/2003.04821

[30] C. Banbury et al., MLPerf Tiny Benchmark, NeurIPS Datasets and Benchmarks (2021). https://arxiv.org/abs/2106.07597

[31] T. Lin et al., MCUNet: Tiny Deep Learning on IoT Devices, NeurIPS (2020). https://arxiv.org/abs/2007.10319

[32] R. David et al., TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems, MLSys (2021). https://arxiv.org/abs/2010.08678

[33] Kaggle, State Farm Distracted Driver Detection competition dataset. https://www.kaggle.com/c/state-farm-distracted-driver-detection

[34] Raspberry Pi Ltd., RP2040 Datasheet. https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf

Appendix A Evidence Inventory and Deduplication Notes

The following notes summarize how older duplicated material was reconciled. The existing short manuscript already contained abstract, introduction, related work, method, hardware deployment, results, conclusion, and five references. The res folder contained newer tables for dataset counts, four-model classification metrics, preliminary transfer-learning metrics, and hardware performance. The newer tables were used as authoritative where overlapping values appeared.

Table 9. Result-source inventory used to construct the manuscript.

Manuscript item	Value or figure	Source file	Traceability note
Dataset table	22,424 images; class counts	res/tables/dataset_class_image_counts.docx	Direct class count table
Driver-wise split	17,382 train / 5,042 validation	distilled_from_*_outputs/driver_wise_split_summary.csv	Same split across four teachers
Four-model metrics	Accuracy/precision/recall/F1	res/tables/Our_Four_Distilled_Model_Accuracy_Precision_Recall_F1.docx	Replaces older repeated table

Manuscript item	Value or figure	Source file	Traceability note
Shared student profile	17 layers, 10 learnable, 0.00553M parameters, 7.81M FLOPs	res/tables/Four_Distilled_Models_Performance_on_Each_Hardware.docx	Mandatory deployment profile
Preliminary selection	Teacher and three student variants	res/tables/Preliminary_Transfer_Learning_Results_and_Deployment_Metrics.docx	Used for Section 8
Hardware timing	2419.6 ms ESP32 profile; 43.4 ms RP2040	res/tables/Representational_Four_Model_Hardware_Performance.docx	Simulator language excluded
Figures	Training, radar, confusion, hardware graphs	res/imgs and model_training_graphs	Placed next to matching discussion

Appendix B Full Four-Student Classification Reports

This appendix preserves the raw class-level validation metrics for all four teacher-derived students. These tables are separated from the main results section so the main paper remains readable while the detailed evidence stays available for audit.

ConvNeXt-derived student

Table 10. Full validation classification report for the ConvNeXt-derived student.

Class/aggregate	Precision	Recall	F1	Support
c0	0.325	0.608	0.424	508.0
c1	0.968	0.354	0.518	517.0
c2	0.412	0.487	0.447	517.0
c3	0.527	0.280	0.366	517.0
c4	0.572	0.595	0.583	509.0
c5	0.993	0.870	0.928	516.0
c6	0.979	0.182	0.306	512.0
c7	0.343	0.566	0.427	470.0
c8	0.225	0.542	0.318	472.0
c9	0.286	0.016	0.030	504.0
accuracy	0.449	0.449	0.449	0.4490281634272114
macro avg	0.563	0.450	0.435	5042.0
weighted avg	0.569	0.449	0.437	5042.0

The ConvNeXt-derived student table is reproduced from C:\Users\HP\Downloads\state-farm-distracted-driver-detection\distilled_from_convnext_outputs\classification_report.csv. Macro and weighted rows are kept because they show how aggregate behavior changes when class support is ignored or included.

EfficientNet-B0-derived student

Table 11. Full validation classification report for the EfficientNet-B0-derived student.

Class/aggregate	Precision	Recall	F1	Support
c0	0.386	0.502	0.436	508.0
c1	0.696	0.803	0.746	517.0
c2	0.461	0.544	0.499	517.0
c3	0.554	0.455	0.499	517.0
c4	0.502	0.267	0.349	509.0
c5	0.980	0.932	0.955	516.0
c6	0.547	0.797	0.649	512.0
c7	0.418	0.483	0.448	470.0
c8	0.333	0.119	0.175	472.0
c9	0.266	0.282	0.274	504.0
accuracy	0.523	0.523	0.523	0.5228084093613645
macro avg	0.514	0.518	0.503	5042.0
weighted avg	0.518	0.523	0.507	5042.0

The EfficientNet-B0-derived student table is reproduced from C:\Users\HP\Downloads\state-farm-distracted-driver-detection\distilled_from_efficientnet_b0_outputs\classification_report.csv. Macro and weighted rows are kept because they show how aggregate behavior changes when class support is ignored or included.

MobileNetV3-Small-derived student

Table 12. Full validation classification report for the MobileNetV3-Small-derived student.

Class/aggregate	Precision	Recall	F1	Support
c0	0.396	0.469	0.429	508.0
c1	0.669	0.662	0.665	517.0
c2	0.499	0.689	0.578	517.0
c3	0.712	0.503	0.590	517.0
c4	0.520	0.629	0.569	509.0
c5	1.000	0.913	0.954	516.0
c6	0.470	0.496	0.482	512.0
c7	0.538	0.547	0.542	470.0
c8	0.400	0.347	0.372	472.0
c9	0.318	0.212	0.255	504.0
accuracy	0.549	0.549	0.549	0.5491868306227687
macro avg	0.552	0.547	0.544	5042.0
weighted avg	0.555	0.549	0.546	5042.0

The MobileNetV3-Small-derived student table is reproduced from C:\Users\HP\Downloads\state-farm-distracted-driver-detection\distilled_from_mobilenet_v3_small_outputs\classification_report.csv. Macro and weighted rows are kept because they show how aggregate behavior changes when class support is ignored or included.

ResNet50-derived student

Table 13. Full validation classification report for the ResNet50-derived student.

Class/aggregate	Precision	Recall	F1	Support
c0	0.493	0.344	0.406	508.0
c1	0.771	0.807	0.788	517.0
c2	0.560	0.747	0.640	517.0
c3	0.452	0.698	0.549	517.0
c4	0.740	0.599	0.662	509.0
c5	0.996	0.884	0.936	516.0
c6	0.771	0.447	0.566	512.0
c7	0.500	0.689	0.580	470.0
c8	0.460	0.367	0.408	472.0
c9	0.348	0.323	0.335	504.0
accuracy	0.593	0.593	0.593	0.5928203094010314
macro avg	0.609	0.591	0.587	5042.0
weighted avg	0.612	0.593	0.590	5042.0

The ResNet50-derived student table is reproduced from C:\Users\HP\Downloads\state-farm-distracted-driver-detection\distilled_from_resnet50_outputs\classification_report.csv. Macro and weighted rows are kept because they show how aggregate behavior changes when class support is ignored or included.

Appendix C Raw Confusion-Matrix Tables

The following confusion matrices are included to make class-level errors traceable. Rows denote true classes and columns denote predicted classes. These matrices support the discussion of c5 stability and the weaker behavior observed for several visually overlapping classes.

ConvNeXt-derived student

Table 14. Raw confusion matrix for the ConvNeXt-derived student.

True	c0	c1	c2	c3	c4	c5	c6	c7	c8	c9
c0	309	0	1	20	71	0	0	86	20	1
c1	40	183	51	1	0	0	0	60	182	0
c2	7	0	252	0	0	0	2	73	183	0
c3	190	0	1	145	139	0	0	41	1	0
c4	99	0	0	82	303	0	0	25	0	0
c5	21	0	4	0	2	449	0	19	21	0
c6	9	4	50	0	0	0	93	20	336	0
c7	13	0	125	2	0	0	0	266	58	6
c8	9	1	114	0	0	0	0	79	256	13
c9	254	1	13	25	15	3	0	106	79	8

EfficientNet-B0-derived student

Table 15. Raw confusion matrix for the EfficientNet-B0-derived student.

True	c0	c1	c2	c3	c4	c5	c6	c7	c8	c9
c0	255	20	0	82	15	0	3	1	3	129
c1	0	415	3	0	0	0	55	26	11	7
c2	0	0	281	0	0	0	134	95	7	0
c3	86	1	2	235	66	3	2	6	0	116
c4	147	12	1	68	136	5	10	1	1	128
c5	4	0	2	14	7	481	1	0	0	7
c6	0	48	33	0	0	0	408	5	18	0
c7	2	12	137	2	1	0	50	227	39	0
c8	1	14	147	0	0	0	82	168	56	4
c9	166	74	3	23	46	2	1	14	33	142

MobileNetV3-Small-derived student

Table 16. Raw confusion matrix for the MobileNetV3-Small-derived student.

True	c0	c1	c2	c3	c4	c5	c6	c7	c8	c9
c0	238	18	2	48	66	0	3	31	3	99
c1	3	342	60	1	0	0	19	59	33	0
c2	3	2	356	0	4	0	138	5	9	0
c3	43	6	11	260	129	0	0	1	14	53
c4	106	1	0	18	320	0	0	3	5	56
c5	3	0	5	0	16	471	0	0	1	20
c6	2	51	106	1	0	0	254	5	93	0
c7	2	5	78	4	3	0	36	257	85	0
c8	8	9	88	1	0	0	89	112	164	1
c9	193	77	8	32	77	0	2	5	3	107

ResNet50-derived student

Table 17. Raw confusion matrix for the ResNet50-derived student.

True	c0	c1	c2	c3	c4	c5	c6	c7	c8	c9
c0	175	1	4	156	24	0	0	12	3	133
c1	3	417	1	4	0	0	38	25	16	13
c2	2	1	386	5	0	0	9	74	38	2
c3	15	6	0	361	69	0	0	1	2	63
c4	12	1	5	156	305	0	0	1	1	28
c5	3	0	0	1	3	456	0	0	0	53
c6	0	67	120	0	0	0	229	30	61	5

True	c0	c1	c2	c3	c4	c5	c6	c7	c8	c9
c7	1	8	82	2	0	0	1	324	49	3
c8	0	34	85	1	0	0	20	154	173	5
c9	144	6	6	112	11	2	0	27	33	163

Appendix D Run Configuration and Deployment Constants

The four final students use matching run configurations. Keeping these values in the manuscript prevents an accidental comparison between different student widths, image sizes, or validation protocols.

Table 18. Distillation run-configuration constants from run_config.json files.

Teacher source	Student size	Channels	Width	Batch	Epoch cap	LR	T	Alpha	Validation
ConvNeXt	96	3	10	32	60	0.001	4.0	0.45	0.2 driver-wise
EfficientNet-B0	96	3	10	32	60	0.001	4.0	0.45	0.2 driver-wise
MobileNetV3-Small	96	3	10	32	60	0.001	4.0	0.45	0.2 driver-wise
ResNet50	96	3	10	32	60	0.001	4.0	0.45	0.2 driver-wise

Table 19. Deployment constants carried into the final manuscript.

Quantity	Value	Source	Interpretation
Input tensor	96 x 96 x 3 INT8	res tables and run_config.json	Shared final student format
Keras layers	17	res deployment profile	Layer count reported in manuscript
Learnable layers	10	res deployment profile	Layers with trainable weights
Parameters	0.00553M	res deployment profile	Reported deployment value
FLOPs	7.81M	res deployment profile	Multiply-add counted as two FLOPs
TFLite size	22,728 bytes	model/student_*_96_rgb_int8.tflite	Same for all four final students
Tensor arena	204,800 bytes	src/main.cpp and res tables	Runtime allocation for TFLite Micro
RP2040 SRAM	264 KB	RP2040 datasheet	Important deployment constraint

The table separates deployment facts from interpretations. It also records the parameter-count caveat: the project source and deployment tables are not silently averaged when they disagree under different reconstruction assumptions.

Appendix E Reference-Topic Map

The references are grouped below by how they support the manuscript. This map is not a substitute for formal citation, but it helps confirm that the reference list is not padded with unrelated works.

Table 20. Citation-use map for the 34-reference bibliography.

References	Topic	Use in manuscript
[1]-[4]	Recent lightweight driver-distraction or driver-state models	Used to position accuracy, FPS, and embedded deployment claims.
[5]-[10]	Recent driver-distraction detection and transfer-learning studies	Used in Related Work and literature benchmarking.
[11]-[15]	Major CNN and backbone families	Used to explain why teacher backbones are powerful but expensive.
[16]-[23]	Efficient convolution and attention modules	Used to justify depthwise separable convolution and compact modeling choices.
[24]-[27]	Distillation, compression, pruning, and quantization	Used to explain teacher-student transfer and INT8 deployment.
[28]-[32]	TinyML and microcontroller benchmarking	Used to motivate tensor-arena and deployment-metric reporting.
[33]-[34]	Dataset and hardware documentation	Used for State Farm class definition and RP2040 memory constraints.

The recent JRTIP references emphasize that lightweight driver monitoring is an active real-time image-processing topic. They report stronger accuracy in many cases, but often on different hardware and with different task formulations.

The backbone references explain why ResNet50, EfficientNet-B0, MobileNetV3-Small, and ConvNeXt are appropriate teacher sources. They are not claimed to be microcontroller deployments in this work.

The TinyML references support the reporting style: parameters, FLOPs, model size, tensor arena, and latency all matter. This is why the manuscript avoids ranking models by accuracy alone.

The dataset and hardware references anchor the empirical setting. The paper uses State Farm for ten-class behavior recognition and RP2040-class SRAM constraints for deployment analysis.

Appendix F Expanded Equation Summary

For convenience, this appendix restates the mathematical quantities used throughout the paper in a compact form. The definitions are included to make the manuscript easier to convert into LaTeX if needed.

Table 21. Equation and metric summary.

Quantity	Formula	Role
Input tensor	X in R^(H x W x C)	Image tensor after resizing and normalization
Convolution	Y(i,j,k)=b_k+sum_m sum_u sum_v W(u,v,m,k)X(i+u,j+v,m)	Spatial feature extraction
Conv parameters	(K_h K_w C_in + 1) C_out	Bias-including standard convolution
Depthwise separable parameters	(K_h K_w C_in + C_in) + (C_in C_out + C_out)	Depthwise spatial filter plus 1x1 pointwise mix
ReLU	max(0,x)	Activation used inside the feature extractor
Max pooling	max over local window Omega	Spatial downsampling

Quantity	Formula	Role
Global average pooling	(1/HW) sum_i sum_j X(i,j,c)	Channel descriptor without flattening
Softmax	exp(z_i)/sum_j exp(z_j)	Class probability conversion
Cross entropy	-log softmax(z_s)_y	Hard-label classification loss
Distillation	alpha CE + (1-alpha)T^2 KL(q_t^T || p_s^T)	Hybrid hard and soft target training
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Overall correctness
Precision	TP/(TP+FP)	Positive predictive reliability
Recall	TP/(TP+FN)	Class coverage
F1	2PR/(P+R)	Harmonic mean of precision and recall
FPS	1000/latency_ms	Derived only for per-frame independent latency values